



Installation Process

Python Interpreter • Python vs Anaconda • IDE Setup • Jupyter Notebook

Before you can write even a single line of Python code, you need to set up the right environment on your computer. Think of this like a **workspace** — a carpenter does not begin cutting wood in an empty room; they first arrange their tools, workbench, and materials. In the same way, a programmer must first install the necessary software so that the computer can understand and run Python code.

This chapter will walk you through every step of that setup process — from installing Python itself to choosing a comfortable coding environment. By the end of this chapter, your computer will be fully ready for Python development.

Chapter Learning Objectives

- Understand what the Python Interpreter is and why it is needed.
- Download and install Python on Windows, macOS, and Linux.
- Understand the difference between standard Python and Anaconda Python.
- Know when to choose which distribution based on your use case.
- Install and configure a professional code editor (VS Code).
- Understand what Jupyter Notebook is and how to use it effectively.

2.1 Python Interpreter Installation

The very first thing you need to understand is: what exactly happens when you run a Python program? The answer lies in something called the Python Interpreter.

Python Interpreter

A program installed on your computer that reads your Python code, understands it, and executes it — one line at a time. Without the interpreter, your computer has no idea what Python code means.

Analogy — The Interpreter in Action

Imagine you have written a letter in French, but your friend only understands English. You would need a **translator** to sit between you and convert each sentence.

The Python Interpreter works the same way. You write code in Python — the interpreter translates it into instructions the computer hardware can actually execute. Without the interpreter, your Python code is just text that the computer cannot act upon.

How the Python Interpreter Works

When you run a Python file, the interpreter follows this sequence:

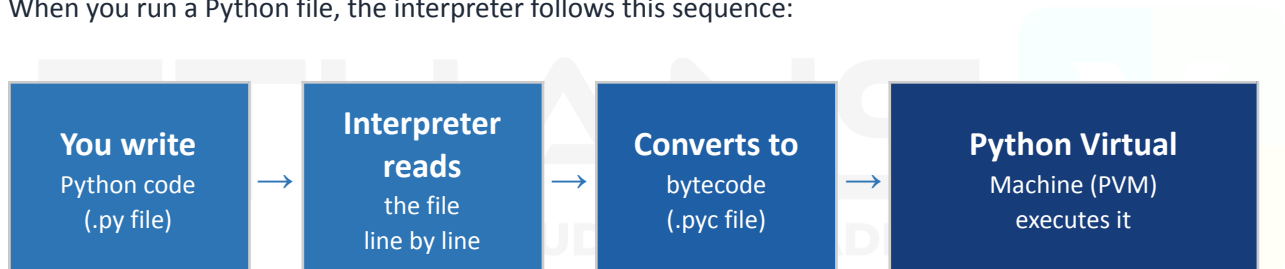


Figure 2.1 — How Python code gets executed by the interpreter

Step-by-Step: Installing Python on Your Computer

Python can be installed on any operating system — Windows, macOS, or Linux. Below is a complete guide for each platform. Follow the steps for your operating system.

Installation on Windows

1. Open your web browser and go to: <https://www.python.org/downloads/> Click the large yellow button labeled **"Download Python 3.x.x"** (the website automatically detects your operating system).
2. Run the downloaded installer file (e.g., **python-3.x.x-amd64.exe**).

Critical Step — Do Not Skip This

On the very first screen of the installer, you will see a checkbox at the bottom that says:

"Add Python to PATH"

You **MUST** check this box before clicking Install. If you skip this step, Python will install but your computer will not be able to find it when you type commands in the terminal. This is the single most common mistake beginners make during installation.

3. Click "**Install Now**" and wait for the installation to complete (usually 1–2 minutes).
4. Once done, click "**Close**".

Verifying the Installation

After installation, you must verify that Python was installed correctly. This is done using the Command Prompt on Windows, or Terminal on macOS/Linux.

On Windows: Press **Windows Key + R**, type **cmd**, and press Enter. Then type the following command and press Enter:

```
python --version
```

Figure 2.2 — Verifying Python installation via Command Prompt

If Python is installed correctly, you will see output like: **Python 3.12.0** (the exact version may differ). If you see an error, the installation did not complete correctly — revisit the steps above.

Installation on macOS

5. Visit <https://www.python.org/downloads/macos/> and download the latest Python 3 installer for macOS.
6. Open the downloaded **.pkg** file and follow the on-screen wizard.
7. Open **Terminal** (search for it in Spotlight with **Cmd + Space**) and run:

```
python3 --version
```

Figure 2.3 — Verifying Python on macOS



Note — macOS and python3

On macOS, you may need to type **python3** instead of **python**. This is because macOS sometimes ships with an older Python 2 installation, and it keeps the two versions separate to avoid conflicts.

Installation on Linux (Ubuntu/Debian)

Most modern Linux distributions come with Python 3 pre-installed. To check:

```
python3 --version
```

Figure 2.4 — Checking Python on Linux

If Python is not installed, open the Terminal and run:

```
sudo apt update  
sudo apt install python3
```

Figure 2.5 — Installing Python on Ubuntu/Debian Linux

 Pro Tip — Always Use Python 3

Always download and use Python 3 (the latest stable version). Python 2 was officially retired in 2020 and is no longer supported. Any resource, course, or library you encounter today is built on Python 3.



2.2 Python vs Anaconda Python

Now that you know how to install Python, you may come across another term — Anaconda. Many data science and machine learning courses recommend Anaconda instead of the standard Python installer. This section explains clearly what Anaconda is, how it differs from standard Python, and how to decide which one is right for you.

What is Standard Python?

Standard Python

The official Python distribution downloaded from python.org. It includes the Python interpreter and a small set of built-in libraries. It is minimal by design — you install only what you need.

When you install Python from python.org, you get:

- The Python interpreter
- The Python standard library (built-in tools like `math`, `os`, `datetime`, etc.)
- `pip` — Python's package installer, used to download and install additional libraries
- A basic interactive shell (REPL)

What is Anaconda Python?

Anaconda

A free, open-source distribution of Python (and R) that is specifically designed for data science, machine learning, and scientific computing. It comes pre-packaged with 250+ popular libraries and its own package manager called `conda`.

Analogy — Standard Python vs Anaconda

Think of Standard Python like buying a **basic smartphone**. It works perfectly, but you need to install your own apps one by one.

Anaconda is like a smartphone that comes **pre-loaded with all the essential apps** a professional would need — email, maps, office tools, camera — right out of the box.

Both are useful. The question is: how much setup time do you want to spend before getting to work?

Key Differences at a Glance

Feature	Standard Python	Anaconda Python
Source / Website	python.org	anaconda.com
Installer Size	~25 MB — very lightweight	~900 MB — large, includes many libraries
Pre-installed Libraries	Minimal (standard library only)	250+ libraries (NumPy, pandas, Matplotlib, etc.)
Package Manager	<code>pip</code>	<code>conda</code> (also supports <code>pip</code>)
Includes Jupyter Notebook?	No — must install manually	Yes — included by default

Feature	Standard Python	Anaconda Python
Includes Spyder IDE?	No	Yes — included by default
Best For	General Python development, web apps, scripting	Data Science, Machine Learning, Scientific Computing
Environment Management	venv (virtual environments)	conda environments (more powerful)
Difficulty for Beginners	Moderate — need to install extras	Easier for data science — everything is ready

Figure 2.6 — Standard Python vs Anaconda: Feature Comparison

Which One Should You Install?

The choice depends on what you plan to do with Python. Here is a simple decision guide:

Your Goal	Recommended Choice
Learn Python basics, web development, automation, scripting	✓ Standard Python from python.org
Data Science, Machine Learning, or AI projects	✓ Anaconda Python
Academic research or scientific computing	✓ Anaconda Python
Building web applications or APIs	✓ Standard Python from python.org
Following along with a data science course	✓ Anaconda Python (most courses assume it)
General purpose learning — not sure yet	✓ Start with Standard Python; switch if needed

Figure 2.7 — Choosing between Standard Python and Anaconda

How to Install Anaconda

If you have decided to use Anaconda, here is how to install it:

- Visit the official Anaconda website: <https://www.anaconda.com/download>
- Download the installer for your operating system (Windows, macOS, or Linux).
- Run the installer and follow all the on-screen instructions. Accept the default settings unless you have a specific reason to change them.
- After installation, open the **Anaconda Navigator** application. This is a graphical interface from which you can launch Jupyter Notebook, Spyder, and other tools.



Note — Anaconda and pip

Anaconda uses its own package manager called **conda**. While you can still use **pip** inside an Anaconda environment, it is generally recommended to use conda when possible, as it handles library compatibility more reliably.



2.3 IDE Installation

You have Python installed on your computer. Now you need a place to write your code. While you can technically write Python in any basic text editor like Notepad, that would be like writing a professional report in crayon — it works, but it is not the right tool for the job.

IDE — Integrated Development Environment

A software application that provides everything a developer needs in one place: a code editor, a way to run code, error highlighting, auto-completion suggestions, and debugging tools. An IDE makes writing code significantly faster and more comfortable.



Analogy — Why Use an IDE?

Imagine writing a 10,000-word research paper in Microsoft Notepad vs. Microsoft Word. Notepad lets you type, but Word adds spell check, formatting, chapter navigation, find-and-replace, and more.

An IDE does the same for code: it catches your mistakes as you type, suggests the next word, lets you run your code with one button, and shows you exactly which line has an error — saving hours of frustration.

Popular IDEs and Code Editors for Python

IDE / Editor	Type	Best For	Free?
Visual Studio Code (VS Code)	Lightweight Code Editor	All-purpose: beginners & professionals	Yes
PyCharm (Community)	Full IDE	Large Python projects, professional use	Yes (Community Edition)
Spyder	Scientific IDE	Data Science (comes with Anaconda)	Yes
Thonny	Beginner IDE	Absolute beginners and students	Yes
Jupyter Notebook	Notebook Interface	Data Science, learning, reporting	Yes

Figure 2.8 — Popular Python IDEs and Code Editors

Recommended Choice: Visual Studio Code (VS Code)

For this course, we recommend Visual Studio Code (VS Code). It is the most widely used code editor in the world, built by Microsoft, and has excellent support for Python through extensions. It is free, lightweight, fast, and used by both beginners and experienced developers at companies like Google, Amazon, and Netflix.

Why VS Code is Recommended for Beginners

- Syntax Highlighting — Different parts of your code appear in different colours, making it easy to read.
- IntelliSense (Auto-complete) — VS Code suggests the next word as you type, reducing errors and saving time.
- Error Underlines — Mistakes are underlined in red as you write, just like a spell checker.

- Integrated Terminal — You can run your Python code without leaving the editor.
- Extensions Marketplace — Thousands of add-ons to customize VS Code for any use case.
- Git Integration — Easily track and manage changes to your code files.

Step-by-Step: Installing VS Code

12. Open your browser and go to: <https://code.visualstudio.com>
13. Click the large **"Download for Windows"** button (or macOS / Linux, depending on your OS). The website detects your operating system automatically.
14. Run the downloaded installer and follow the setup wizard. Accept all default settings.
15. During installation, check the following options when presented:
 - **"Add to PATH"** — Allows you to open VS Code from the terminal.
 - **"Register Code as an editor for supported file types"**
16. Click **"Install"** and then **"Finish"** when complete. VS Code will open automatically.

Installing the Python Extension for VS Code

VS Code on its own is a general-purpose editor. To get full Python support (auto-complete, error detection, the ability to run Python files), you need to install the Python extension created by Microsoft.

17. Open VS Code.
18. On the left sidebar, click the **Extensions icon** (it looks like four small squares, or press **Ctrl+Shift+X**). In the search bar, type **"Python"**.
19. The first result should be the **"Python"** extension by **Microsoft** (it will show a blue Microsoft badge). Click **"Install"**.
20. Once installed, VS Code may ask you to select a Python interpreter. Click on the notification and choose the Python version you installed from python.org or Anaconda.

Writing and Running Your First Python File in VS Code

21. Go to **File > New File** and save it as **hello.py** (the .py extension tells VS Code this is a Python file).
22. Type the following code:

```
print("Hello, World!")
print("My Python environment is ready.")
```

Figure 2.9 — First Python program in VS Code

23. To run the file, click the **Run button** (the triangle ► icon in the top-right corner) or press **Ctrl+F5**.
24. The output appears in the **Terminal** panel at the bottom of VS Code:

```
Hello, World!
My Python environment is ready.
```

Figure 2.10 — Output of the first Python program in VS Code terminal



Pro Tip — Keyboard Shortcut to Run Python

In VS Code, you can run any Python file by pressing **Ctrl+F5** (Windows/Linux) or **Cmd+F5** (macOS). This runs the file without a debugger — which is the most common way to test your code.



2.4 Introduction to Jupyter Notebook

So far, you have learned to write Python in a traditional code editor like VS Code where you write the entire program first and then run it. Jupyter Notebook offers a completely different — and for many tasks, a superior — way of working with Python.

Jupyter Notebook

An interactive, web-based coding environment that allows you to write Python code in small, runnable sections called 'cells'. Each cell can be run individually, and the output appears directly below it. You can mix code, formatted text, images, and charts in a single document.



Analogy — Notebook vs Script

Think of a traditional Python script (.py file) as writing a full recipe on one sheet of paper and then cooking the entire meal in one go. If something goes wrong on step 7, you have to start over from step 1.

Jupyter Notebook is like having a **spiral-bound recipe notebook** where you cook one step, taste-test the result, then move to the next. Each step is independent. You can re-run step 3 without affecting steps 1, 2, 4, or 5. This makes experimenting and learning much more efficient.

Why Jupyter Notebook is Important

Jupyter Notebook is the industry standard tool in data science and machine learning. When data scientists and analysts share their work, they share it as Jupyter Notebooks — because it shows the code, the results, and the explanation all in one place, side by side.

Feature	VS Code (.py file)	Jupyter Notebook (.ipynb file)
How code is written	All code in one file, runs top to bottom	Code split into cells, each runs independently
Output display	Shown in terminal at the bottom	Shown directly below each cell
Charts and graphs	Pop up in a separate window	Displayed inline, inside the notebook
Text and explanations	Only as code comments (#)	Full formatted text using Markdown
Best use case	Scripts, apps, automation, web backends	Data analysis, learning, research, reporting
File format	.py (plain Python script)	.ipynb (JSON-based notebook format)
Execution style	All at once (or debugged step by step)	Cell by cell — selective and interactive

Figure 2.11 — VS Code vs Jupyter Notebook: A Comparison

Installing Jupyter Notebook

Jupyter Notebook can be installed in two ways depending on your Python setup:

Option A: If You Have Standard Python (python.org)

Open your Command Prompt (Windows) or Terminal (macOS/Linux) and run:

```
pip install notebook
```

Figure 2.12 — Installing Jupyter Notebook using pip

Wait for the installation to complete. Once done, you can launch Jupyter by running:

```
jupyter notebook
```

Figure 2.13 — Launching Jupyter Notebook from the terminal

Option B: If You Have Anaconda

Jupyter Notebook comes pre-installed with Anaconda. You have two ways to launch it:

- Through Anaconda Navigator: Open Anaconda Navigator > Click Launch under Jupyter Notebook.
- Through Terminal/Command Prompt: Type **jupyter notebook** and press Enter.



Note — JupyterLab vs Jupyter Notebook

You may also come across **JupyterLab**, which is the next-generation interface for Jupyter. It offers a more powerful, flexible workspace with multiple notebooks open at once, a file browser, and more tools. For this course, the classic Jupyter Notebook is sufficient. You can install JupyterLab later with: **pip install jupyterlab**

Understanding the Jupyter Interface

When you run jupyter notebook in your terminal, it opens a tab in your default web browser. This is the Jupyter Dashboard.

From the dashboard, you can:

- Navigate your computer's folders and files
- Create a new notebook by clicking **New > Python 3 (ipykernel)**
- Open an existing notebook by clicking on its name
- Upload files from your computer

Anatomy of a Jupyter Notebook

Once you open a new notebook, you will see the notebook editor. Here is what each part does:

Component	What It Does
Menu Bar	Contains File, Edit, View, Insert, Cell, Kernel menus for all notebook operations.
Toolbar	Quick-access buttons: Save, Add Cell, Cut/Copy/Paste Cell, Run Cell, Stop, Restart Kernel.
Code Cell	Where you type Python code. Press Shift+Enter to run the cell and move to the next one.
Output Area	The result of running a code cell appears directly below it.

Component	What It Does
Kernel Indicator	Shows which Python environment is active. In [*] means the cell is currently running.
Cell Execution Count	The number in [] next to a cell shows the order in which it was run. [3] means it was the 3rd cell executed.

Figure 2.14 — Components of the Jupyter Notebook interface

Cell Types in Jupyter Notebook

A Jupyter Notebook is made up of cells. There are two main types of cells you need to know:

Cell Type	Purpose	Example
Code Cell	Contains Python code. When run, the output appears below the cell.	<code>print("Hello, World!")</code>
Markdown Cell	Contains formatted text, headings, explanations, images. Does not run code.	<code>## This is a Section Title</code> <code>**Bold text**</code> , <code>*italic text*</code>

Figure 2.15 — Two main cell types in Jupyter Notebook

Essential Keyboard Shortcuts in Jupyter Notebook

Jupyter Notebook has two modes — Command Mode (navigating between cells) and Edit Mode (typing inside a cell). Knowing the key shortcuts makes working in Jupyter significantly faster.

Shortcut	Action	Mode
Shift + Enter	Run the current cell and move to the next cell	Both
Ctrl + Enter	Run the current cell and stay on the same cell	Both
A	Insert a new cell above the current cell	Command Mode
B	Insert a new cell below the current cell	Command Mode
D, D (press D twice)	Delete the current cell	Command Mode
M	Change the cell type to Markdown	Command Mode
Y	Change the cell type to Code	Command Mode
Esc	Exit Edit Mode and enter Command Mode	Edit Mode
Enter	Enter Edit Mode in the selected cell	Command Mode
Ctrl + S	Save the notebook	Both

Figure 2.16 — Keyboard shortcuts for Jupyter Notebook

Your First Jupyter Notebook — A Practical Walkthrough

Let us create a real Jupyter Notebook step by step to see how it all works in practice.

25. Launch Jupyter Notebook from your terminal by typing **jupyter notebook** and pressing Enter. A browser tab will open.
26. Click **New > Python 3 (ipykernel)** in the top-right corner. A new notebook opens.
27. Click on the title "**Untitled**" at the top and rename it to **my_first_notebook**.
28. In the first cell (a Code cell), type the following:

```
# This is my first Jupyter Notebook
print("Hello from Jupyter!")
print(2 + 3)
```

29. Press **Shift + Enter** to run the cell. The output appears directly below:

```
Hello from Jupyter!
5
```

Figure 2.17 — Running a code cell in Jupyter Notebook — output appears below

30. Press **B** (in Command Mode) to add a new cell below. Then press **M** to change it to a Markdown cell and type:

```
## My Notes
This notebook demonstrates basic Python operations.
The next cell will perform arithmetic.
```

31. Press **Shift + Enter**. The Markdown cell renders as formatted text, not code.



Where Jupyter Notebooks Are Used Professionally

Jupyter Notebooks are the standard format for:

- Data Science reports — analysts share notebooks that show their analysis step by step.
- Machine Learning experiments — researchers document model training, results, and observations.
- Academic research papers — scientists publish reproducible research as notebooks.
- Business dashboards — teams share data-driven insights with embedded charts.
- Online courses and tutorials — platforms like Coursera, Kaggle, and Google Colab use notebook format for lessons.

Chapter Summary

- **The Python Interpreter** is the program that reads your Python code and executes it. Without it, Python code is just text that the computer cannot understand.
- **Python can be installed** on Windows, macOS, and Linux from python.org. The 'Add Python to PATH' option during Windows installation is critical.
- **Standard Python (python.org)** is lightweight and suitable for general development, web apps, and scripting.
- **Anaconda Python (anaconda.com)** is a full distribution pre-loaded with 250+ data science libraries. It is the preferred choice for data science and machine learning.
- **An IDE** makes writing code faster and more comfortable. VS Code is the recommended choice — it is free, powerful, and widely used in the industry.

- **The Python Extension for VS Code** must be installed separately to enable Python-specific features like IntelliSense, error detection, and code running.
- **Jupyter Notebook** is an interactive, cell-based coding environment where code, output, and formatted text coexist in one document.
- **Cells in Jupyter** can be Code cells (Python code) or Markdown cells (formatted text). Each code cell is run independently using Shift+Enter.

